Programming 1

Laszlo SZATHMARY
University of Debrecen
Faculty of Informatics

Lecture #4

- what is evaluated as False; string buffer
- tuple data type
- list comprehension
- control structures
- functions

(last update: 2026-01-17 [yyyy-mm-dd])

2025-2026, 2nd semester

What is evaluated as False?

False
None
0 # and: 0.0
“ # or: ”””
[]
()
{}

empty sequences

empty dictionary

```
>>> bool(None)
False
>>> bool([])
False
>>> bool([1,2,3])
True
>>> bool("py")
True
```

Anything else is considered to to be True.

HW: XOR.

http://docs.python.org/3/library/stdtypes.html#truth-value-testing

String buffer

Example: consider the natural numbers from 1 to 15 (included) and write them down one after the other. Let the result be a string: „`123456789101112131415`”.

String buffer

Example: consider the natural numbers from 1 to 15 (included) and write them down one after the other. Let the result be a string: „`123456789101112131415`”.

Naïve approach:

```
res = ""
for i in range(1, 15+1):
    res += str(i)
#
print(res)
```

Using a string buffer:

```
parts = []
for i in range(1, 15+1):
    parts.append(str(i))
#
res = "".join(parts)
print(res)
```

tuple

```
>>> a = (1, 2, 3)
>>> a[0]
1
>>> a[0] = 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>>
>>> m = ('Total Recall', 1990, 7.5)
>>> m
('Total Recall', 1990, 7.5)
>>> len(m)
3
>>> m[:2]
('Total Recall', 1990)
>>>
>>> (x, y) = (1, 2)
>>> x
1
>>> y
2
>>> single = ('hi',)
>>> single
('hi',)
```

immutable (read-only)

parallel assignment (parentheses are often optional)

tuple with one element (Notice the special syntax!)

HW: tuple02.py

```python
def get_movie_info():
    # contact database,
    # fetch info from a website, etc.
    return ('Total Recall', 1990, 7.5)


def main():
    (title, year, score) = get_movie_info()
    print('Title:', title)
    print('Year:', year)
    print('Score:', score)
```

using this method, a function can return **multiple values**

value unpacking

```
>>> a = 3
>>> b = 9
>>> a
3
>>> b
9
>>> a, b = b, a
>>> a
9
>>> b
3
>>>
```

Swap two variables.
No temp variable is needed :)

List comprehension: a compact way for building lists

```
>>> nums = [1, 2, 3, 4]
>>> squares = [n*n for n in nums]
>>> squares
[1, 4, 9, 16]
```

```
>>> nums = [1, 2, 3, 4]
>>> squares = []
>>> for n in nums:
...     squares.append(n*n)
...
>>> squares
[1, 4, 9, 16]
```

Generally:

[*expr* for *var* in *list*]

Optional „if”:

```
>>> nums = [8, 3, 2, 1, 5, 9, 2]
>>> small = [n for n in nums if n <= 2]
>>> small
[2, 1, 2]
>>>
```

it keeps only those elements that satisy the condition

Exercise

Solve the following exercises with list comprehensions.

Link: https://arato.inf.unideb.hu/szathmary.laszlo/pmwiki/index.php?n=EnPy3.20120818d

Common operation: reverse a sequence

```
>>> s = 'Python rulz'
>>> s[::-1]
'zlur nohtyP'
>>>
>>> li = [23, 76, 84, 12]
>>> li[::-1]
[12, 84, 76, 23]
>>>
```

from the beginning to the end,
but with a reversed step
(it returns a new sequence)

Raising to a power; int and long types

2

```
>>> 2**3
8
>>> 2**64
18446744073709551616L
>>> 2**128
340282366920938463463374607431768211456L
>>>
>>> import sys
>>> sys.maxint
9223372036854775807
>>> a = sys.maxint + 1
>>> a
9223372036854775808L
>>> type(sys.maxint)
<type 'int'>
>>> type(a)
<type 'long'>
```

Python can handle arbitrarily
large integers
(only depends on the memory)

here: 64 bit int (2**63 - 1)

instead of an overflow, its type
is changed automatically from
„int” to „long”

Python 3: there is only *int* type, but in the case of big numbers it behaves like *long* .

if / elif / else

```
4   def main(num):
5       if num < 0:
6           print('Negative number.')
7       elif num == 0:
8           print('Zero.')
9       elif num < 100:
10          print('Less than 100.')
11      else:
12          print('More than 100.')    # >= 100, actually
```

Instead of „else if” we write „elif”.

How to memorize that? The word „elif” is just as long as the word „else”…

for

```
4   def main():
5       li = ['alfa', 'beta', 'gamma']
6
7       for e in li:
8           print(e)
9
10      for index, e in enumerate(li):
11          print(index, e)
```

alfa
beta
gamma

0 alfa
1 beta
2 gamma

It's a common case that we also need the indexes of the elements.
Use the `enumerate()` function, which is also an iterator in Python 3.

See also `enumerate(my_list, start=1)`.

break / continue
pass

```
4   def main():
5       cnt = 0
6       while True:
7           cnt += 1
8           if cnt == 42:
9               break
10      #
11      print(cnt)    # 42
12
13      li = ['ananas', 'banana', 'orange']
14      for e in li:
15          if e == 'banana':
16              continue
17          #
18          print(e)
19
20
21  def palindrome(s):
22      pass     # TODO...
```

42

ananas
orange

empty statement

docstring

Documentation should be one line:

```
def square(num):
    """Return the square of a given number."""
    return num ** 2
```

Or multiline. In this case the first line should be a short summary in one line. Then leave a line empty, and then you can detail the goal of the function, how it works, what are its side effects, etc.

```
def square_v2(num):
    """Return the square of a given number.

    Calculate the square of the input number."""
    return num * num
```

(see also: annex F)

<u>Document your programs! Get used to it!</u>

optional parameters / default parameter values

```
>>> def greet(name, greetings="Hello"):
...     print("{g}, {n}!".format(g=greetings, n=name))
...
>>> greet("Laszlo")
Hello, Laszlo!
>>>
>>> greet("Laszlo", greetings="Hola")
Hola, Laszlo!
>>>
>>> greet("Laszlo", "Bonjour")
Bonjour, Laszlo!
```

optional parameters (another example)

```
4   def hello(name, repeat=1, postfix=''):
5       for i in range(repeat):
6           print(name + postfix)
7
8
9   def main():
10      hello('Laci')
11      print('#' * 10)
12      hello('Laci', repeat=3)
13      print('#' * 10)
14      hello('Laci', postfix='!')
15      print('#' * 10)
16      hello('Laci', repeat=3, postfix='!')
17      print('#' * 10)
18      print('#' * 10)
19      hello('Laci', 3)
20      print('#' * 10)
21      hello('Laci', 3, '')
22      print('#' * 10)
23      hello('Laci', '')      # error
```

homework

Exercises

1. [20120818a] distance between two points (tuple)
2. [20120818d] list comprehensions (to complete)
3. [20120818e] multiples of 3 or 5 (PE #1)
   (this time with list comprehensions)
4. [20120920e] line break
5. [20130211b] diamond
6. [20120818f] sum square difference (PE #6)
7. [20130305a] XOR (Warning! Requires thinking!)